

Chapter 6 Stacks

Stack Abstract Data Type (ADT)

◆ Prompt 1: What is a Stack Abstract Data Type (ADT)?

✓ Answer:

A **stack** is an **abstract data type (ADT)** that stores elements in a **Last In, First Out (LIFO)** order.

This means:

- The **last element added** to the stack is the **first one removed**.

Stacks are simple but very important and are used in:

- Function calls (call stack)
- Undo/redo operations
- Expression evaluation
- Algorithm design (DFS, backtracking)

◆ Prompt 2: What does “abstract data type” mean?

✓ Answer:

An **abstract data type (ADT)** defines:

- **What operations** can be performed
- **What those operations do**

But it does **not** specify:

- How the data is stored internally
- How the operations are implemented

For a stack, we care about *push*, *pop*, *top*, etc., not whether it's implemented using arrays or linked lists.

◆ Prompt 3: What are the main stack operations?

✓ Answer:

A stack supports two main methods:

push(e)

- Adds element e to the **top** of the stack.

pop()

- Removes and returns the **top element**.
- Causes an **error if the stack is empty**.

◆ Prompt 4: What are the accessor (helper) methods of a stack?

✓ Answer:

Accessor methods do not modify the stack:

- **top()**
Returns the top element **without removing it**.
Error if stack is empty.
- **isEmpty()**
Returns `True` if the stack has no elements.
- **len(S)**
Returns the number of elements in the stack.

◆ Prompt 5: What assumptions are made about a stack?

✓ Answer:

By convention:

- A newly created stack is **empty**
- There is **no fixed capacity limit**
- Stack elements can be of **any data type**

◆ Prompt 6: What is an error condition in a stack?

✓ Answer:

An **error** occurs when:

- `pop()` is called on an **empty stack**
- `top()` is called on an **empty stack**

This is called **stack underflow**.

◆ Prompt 7: Explain the example stack operations step by step

✓ Answer:

Operation	Return Value	Stack Contents
push(5)	—	[5]
push(3)	—	[5, 3]
len(S)	2	[5, 3]
pop()	3	[5]
isEmpty()	False	[5]
pop()	5	[]
isEmpty()	True	[]
pop()	error	[]
push(7)	—	[7]
push(9)	—	[7, 9]
top()	9	[7, 9]
push(4)	—	[7, 9, 4]
len(S)	3	[7, 9, 4]
pop()	4	[7, 9]
push(6)	—	[7, 9, 6]
push(8)	—	[7, 9, 6, 8]
pop()	8	[7, 9, 6]

◆ Prompt 8: How does LIFO work in a stack?

✓ Answer:

LIFO means:

Last element pushed → first element popped

Example:

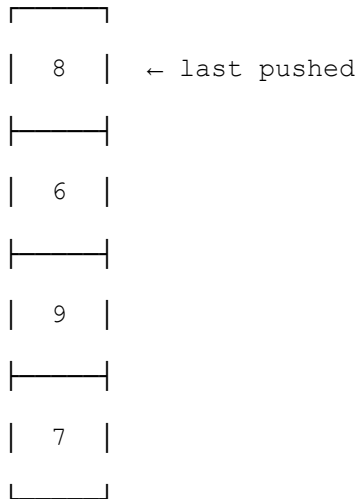
Push order: 5 → 3 → 9

Pop order: 9 → 3 → 5

◆ Prompt 9: Draw a stack diagram

✓ Answer (ASCII Image):

Top



Bottom

◆ Prompt 10: Write a simple stack implementation in Python

✓ Answer:

```
class Stack:

    def __init__(self):

        self.data = []

    def push(self, e):

        self.data.append(e)

    def pop(self):

        if self.is_empty():

            raise IndexError("Stack is empty")

        return self.data.pop()

    def top(self):

        if self.is_empty():

            raise IndexError("Stack is empty")

        return self.data[-1]

    def is_empty(self):

        return len(self.data) == 0

    def __len__(self):

        return len(self.data)
```

◆ Prompt 11: What are common real-world uses of stacks?

✓ Answer:

- Function calls (call stack)
- Undo/redo in editors
- Expression evaluation
- Parentheses checking
- Backtracking algorithms
- Depth-First Search (DFS)

Array-Based Stack Implementation

◆ Prompt 1: What is an array-based stack implementation?

✓ Answer:

An **array-based stack implementation** stores stack elements in a **linear array structure**. In Python, this is naturally implemented using a **list** because:

- Lists allow adding elements to the end using `append()`
- Lists allow removing elements from the end using `pop()`

The **top of the stack** is aligned with the **rightmost element** of the list.

◆ Prompt 2: Why is the top of the stack stored at the end of the list?

✓ Answer:

Placing the stack's top at the **end of the list** is efficient because:

- `append()` runs in **$O(1)$** amortized time
- `pop()` runs in **$O(1)$** time
- No shifting of elements is required

This matches the **LIFO (Last-In, First-Out)** behavior of stacks.

◆ Prompt 3: Why shouldn't we use Python lists directly as stacks?

✓ Answer:

Although Python lists *can* act as stacks, using them directly is **not ideal** because:

- Lists allow inserting/removing elements **from any position**
- This breaks the **stack abstraction**
- The list interface uses terms like `append`, not `push`

A stack should **only allow access to the top element**.

In this case:

- Python's **list** is adapted to behave like a **stack ADT**

◆ Prompt 4: How is the adapter pattern applied to stacks?

✓ Answer:

We:

1. Store a Python list as a **private attribute**
2. Implement stack methods using list methods
3. Hide list operations that break stack rules

This preserves the **stack abstraction** while reusing Python's list.

◆ Prompt 5: What is the mapping between stack operations and list operations?

✓ Answer (Table Explanation):

Stack Operation	Python List Operation
<code>push(e)</code>	<code>append(e)</code>
<code>pop()</code>	<code>pop()</code>
<code>top()</code>	<code>L[-1]</code>
<code>isEmpty()</code>	<code>len(L) == 0</code>
<code>len(S)</code>	<code>len(L)</code>

◆ Prompt 6: Write a Python stack using the adapter pattern

✓ Answer (Code):

```
class ArrayStack:

    def __init__(self):

        self._data = []    # internal list

    def push(self, e):

        self._data.append(e)

    def pop(self):

        if self.is_empty():

            raise IndexError("Stack is empty")

        return self._data.pop()

    def top(self):
```

```
    if self.is_empty():  
        raise IndexError("Stack is empty")  
    return self._data[-1]  
  
def is_empty(self):  
    return len(self._data) == 0  
  
def __len__(self):  
    return len(self._data)
```

◆ Prompt 7: What are the advantages of array-based stacks?

✓ Answer:

- Simple implementation
- Efficient `push` and `pop`
- Uses built-in Python list
- No need for pointer manipulation
- Good cache performance

◆ Prompt 8: What are the limitations?

✓ Answer:

- Resizing may occur (though amortized efficient)
- Not suitable if strict memory control is needed
- Less flexible than linked-list stacks for certain cases

Matching Parentheses and Delimiters Using Stacks

◆ Prompt 1: What is the matching delimiters problem?

✓ Answer:

The **matching delimiters problem** checks whether symbols that come in **pairs** are **properly opened and closed** in an expression.

Common delimiter pairs:

- Parentheses: ()
- Braces: { }
- Brackets: []

Each **opening symbol** must:

1. Be matched with the **correct closing symbol**
2. Be closed in the **correct order**

◆ Prompt 2: What does “properly matched” mean?

✓ Answer:

Delimiters are **properly matched** if:

- Every opening symbol has a corresponding closing symbol
- Symbols close in the **reverse order** in which they open
- No closing symbol appears before its matching opening symbol

◆ Prompt 3: Give examples of correct and incorrect expressions

✓ Answer:

✓ *Correct expressions:*

() (()) { ([()]} }

((() (()) { ([()]})))

[(5+x) - (y+z)]

✗ *Incorrect expressions:*

) (()) { ([()]) } ← closing before opening

{ ([]) } ← mismatched symbols

(← opening without closing

◆ Prompt 4: Why are stacks ideal for this problem?

✓ Answer:

Stacks follow **Last-In, First-Out (LIFO)** order, which perfectly matches how delimiters work:

- The **most recent opening symbol** must be closed **first**
- This behavior is exactly what a stack provides

◆ Prompt 5: What is the main idea of the algorithm?

```
1 def is_matched(expr):
2     """Return True if all delimiters are properly match; False otherwise."""
3     lefty = '({[' # opening delimiters
4     righty = ')}]' # respective closing delims
5     S = ArrayStack()
6     for c in expr:
7         if c in lefty:
8             S.push(c) # push left delimiter on stack
9         elif c in righty:
10            if S.is_empty():
11                return False # nothing to match with
12            if righty.index(c) != lefty.index(S.pop()):
13                return False # mismatched
14    return S.is_empty() # were all symbols matched?
```

Code Fragment 6.4: Function for matching delimiters in an arithmetic expression.

✓ Answer:

1. Scan the expression **left to right**
2. Push every **opening delimiter** onto a stack
3. When a **closing delimiter** is found:
 - Pop from the stack
 - Check if it matches

At the end:

- If the stack is empty → expression is valid
- Otherwise → expression is invalid

◆ Prompt 6: Write the delimiter-matching algorithm in Python

✓ Answer (Code):

```
def ismatched(expr):  
  
    """Return True if all delimiters are properly matched; False  
    otherwise."""  
  
    lefty = '({['      # opening delimiters  
    righty = ')}]}'    # closing delimiters  
    S = []              # stack implemented as list  
  
    for c in expr:  
        if c in lefty:  
            S.append(c)          # push opening symbol  
        elif c in righty:  
            if not S:  
                return False      # nothing to match  
            if righty.index(c) != lefty.index(S.pop()):  
                return False      # mismatched pair  
  
    return not S                # stack should be empty
```

◆ Prompt 7: Explain the algorithm step by step using an example

✓ **Answer:**

Expression:

[(5+x) - (y+z)]

Step-by-step processing:

Character	Action	Stack
[	push	[]
(	push	[[, (]
)	pop (	[]
(	push	[[, (]
)	pop (	[]
]	pop [	[]

✓ Stack is empty → **expression is valid**

◆ Prompt 8: What conditions cause the algorithm to return False?

✓ **Answer:**

The algorithm fails if:

1. A closing delimiter appears when the stack is empty
2. The closing delimiter does not match the last opening delimiter
3. The stack is not empty at the end

◆ Prompt 9: What is the time complexity of the algorithm?

✓ Answer:

Let n be the length of the expression:

- Each character is processed once $\rightarrow O(n)$
- Each push/pop is $O(1)$
- Lookup operations are constant time

🕒 Final Time Complexity:

$O(n)$

Matching Tags in a Markup Language (HTML/XML)

◆ Prompt 1: What is a markup language?

✓ Answer:

A **markup language** is a system for annotating text so that it can be processed and displayed correctly.

Examples:

- **HTML** (HyperText Markup Language) $\rightarrow$ web pages
- **XML** (Extensible Markup Language) $\rightarrow$ structured data

Markup languages use **tags** to define structure and meaning.

◆ Prompt 2: What are HTML tags?

✓ Answer:

HTML tags are **delimiters** that mark sections of content.

- **Opening tag:** `<tagname>`
- **Closing tag:** `</tagname>`

Example:

```
<p> This is a paragraph </p>
```

Here:

- <p> opens the paragraph
- </p> closes the paragraph

◆ Prompt 3: What does it mean for HTML tags to be properly matched?

✓ Answer:

HTML tags are **properly matched** if:

1. Every opening tag has a corresponding closing tag
2. Tags are closed in the **correct order**
3. Tags are **properly nested**

Correct:

```
<body>  
  <p>Text</p>  
</body>
```

Incorrect:

```
<body>  
  <p>Text</body>  
</p>
```

◆ Prompt 4: Why are stacks suitable for matching HTML tags?

✓ Answer:

Stacks follow **Last-In, First-Out (LIFO)** order, which matches HTML nesting rules:

- The **most recently opened tag** must be the **first one closed**
- This nesting behavior maps perfectly to stack operations

◆ Prompt 5: What is the main idea of the HTML tag-matching algorithm?

✓ Answer:

1. Scan the HTML document **from left to right**
2. When an **opening tag** is found → push it onto a stack
3. When a **closing tag** is found → pop from the stack and compare
4. If mismatched or stack is empty → invalid HTML
5. At the end → stack must be empty for valid HTML

◆ Prompt 6: How does the algorithm find HTML tags in the text?

✓ Answer:

- It searches for < to locate the start of a tag
- It searches for > to locate the end of a tag
- The tag name is extracted from between these symbols

Example:

```
<h1>
```

Extracted tag:

```
h1
```

◆ Prompt 7: Write the HTML tag-matching algorithm in Python

✓ Answer (Code):

```
def ismatchedhtml(raw):  
    """Return True if all HTML tags are properly matched; False otherwise."""  
    S = []  
    j = raw.find('<')          # find first '<'  
  
    while j != -1:  
        k = raw.find('>', j + 1) # find matching '>'  
        if k == -1:  
            return False        # invalid tag  
        tag = raw[j + 1:k]      # extract tag name  
        if not tag.startswith('/'): # opening tag  
            S.append(tag)  
        else:                    # closing tag  
            if not S:  
                return False  
            if tag[1:] != S.pop():  
                return False  
        j = raw.find('<', k + 1) # find next '<'  
  
    return not S                # stack must be empty
```

◆ Prompt 8: Explain the algorithm step by step using an example

✓ Answer:

HTML snippet:

```
<body>

  <p>Hello</p>

</body>
```

Step-by-step execution:

Tag Found	Action	Stack
<body>	push	[body]
<p>	push	[body, p]
</p>	pop p	[body]
</body>	pop body	[]

✓ Stack empty → **valid HTML**

◆ Prompt 9: What errors can the algorithm detect?

✓ Answer:

The algorithm returns **False** if:

1. A closing tag appears with no opening tag
2. Tags do not match (e.g., <p> closed by </div>)
3. A tag is missing >
4. Stack is not empty at the end

◆ Prompt 10: What is the time complexity of the algorithm?

✓ Answer:

Let n be the number of characters in the HTML document.

- Each character is scanned at most once
- Stack operations are constant time

🕒 Time Complexity:

$O(n)$